

PROJET DOOM-LIKE P2P

COMPTE-RENDU D'AVANCEMENT : FIN DE SEMESTRE 5

Équipe : Chailan Cyprian, Gros Geoffrey, Lounici Ilyes,
Ray Marcelin
13 fevrier 2026

Sommaire

- Objectifs du Projet
- Cahier des Charges
- Le Moteur Graphique
- Algorithmes liés au déplacement des monstres
- Architecture Réseau
- Bilan et Vision Finale

Objectifs du Projet

- **But** : Ne pas créer un jeu commercial complet, mais développer un **prototype technique** pour explorer les fondements d'un moteur de jeu
- **Inspirations** : *Wolfenstein 3D* (pour le Raycasting) et *Doom* (pour le BSP)
- **Contraintes techniques** : Langage Java (Swing pour le rendu, Sockets pour le réseau) sans utiliser de moteur existant (ex: Unity/Unreal).



Wolfenstein 3D (1992)



DOOM (1993)

Cahier des Charges

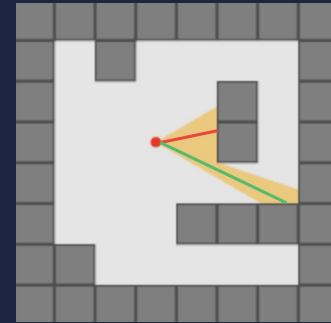
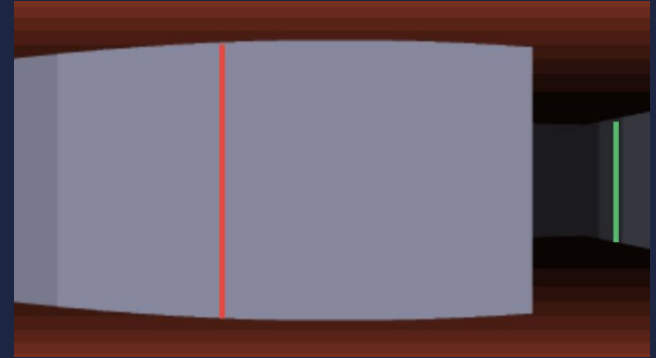
- **Graphisme** : Moteur "2.5D" (Raycasting), Moteur basé sur du BSP, textures, sprites
- **Comportement PNJ** : Ennemis avec Pathfinding (RRT*, Steering Behaviors) et comportements (Automates)
- **Réseau** : Multijoueur Peer-to-Peer (P2P) pour la synchronisation des états.

BSP (Partition Binaire de l'Espace) : est une méthode pour diviser un espace de manière récursive et ordonner les objets au sein de cet espace.

Etat à la fin du premier semestre : Le Moteur Graphique

Le Raycasting (la base) :

- **Principe** : Projection d'un environnement 2D en pseudo-3D. Lancer de rayons depuis le joueur pour calculer la distance des murs
- **Problématique initiale** : La détection naïve (avancer pas à pas) est coûteuse ou imprécise
- **Solution utilisée** : Algorithme DDA (Digital Differential Analysis) pour scanner la grille efficacement.



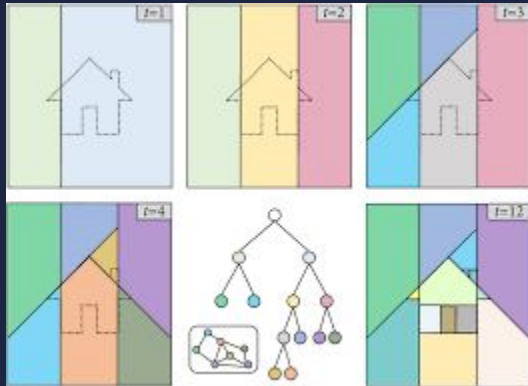
https://en.wikipedia.org/wiki/Ray_casting

Etat à la fin du premier semestre : Le Moteur Graphique

L'Optimisation BSP (Binary Space Partitioning) :

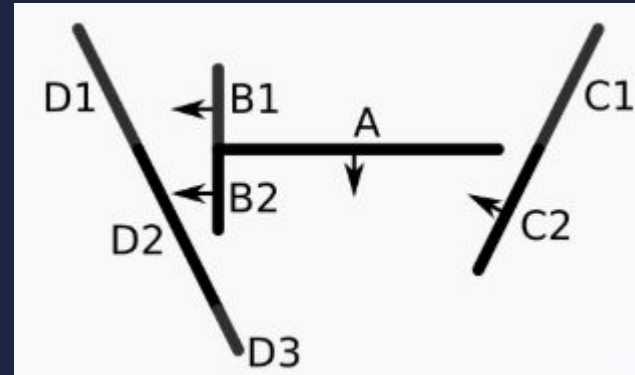
- **Pourquoi le BSP ?** Pour optimiser le rendu en ne traitant que ce qui est nécessaire, contrairement au Raycasting brut qui peut ralentir sur de grandes maps.
- **Fonctionnement de notre implémentation :** Découpage récursif de la carte en un arbre binaire, puis navigation dans ce dit arbre.

imprévu - Mauvais BSP



https://www.researchgate.net/figure/illustration-of-adaptive-binary-space-partitioning-During-partitioning-a-binary-tree-is_fig1_372416349

Doom : BSP room/portail



https://fr.wikipedia.org/wiki/Partition_binaire_de_l'espace

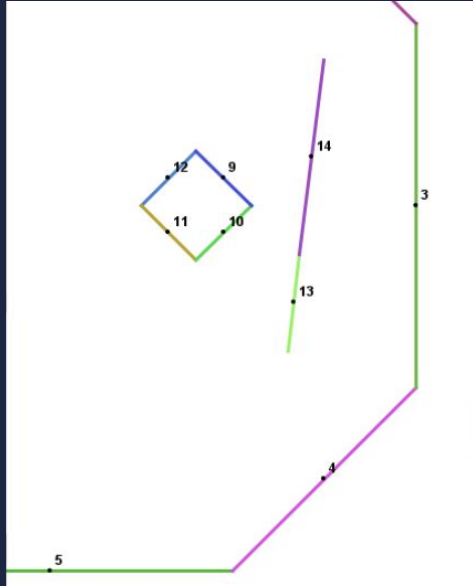
Nous : BSP segment

imprévu - Mauvais BSP

- Beaucoup de confusion entre les différentes sources, le BSP est très général et ne reflète pas un algorithme précis.
- Notre implémentation différente implique un parcours et découpage de l'arbre différent. (réflexion par “segment” de mur, et non par “salle” convexe liée par des portails)
- Elle ne nous empêche pas d'avoir des rendus prometteurs avec notre implémentation, d'optimiser la vitesse d'affichage, la résolution, le temps de calcul...

Ce que l'on a déjà fait ? - BSP

Découpage de la map

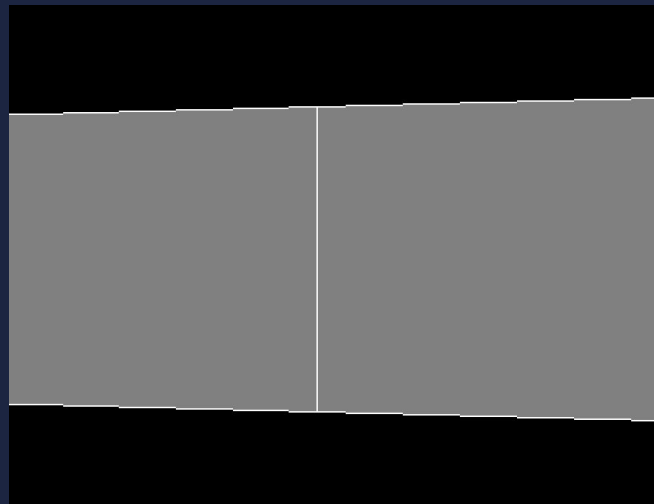


Parcours de l'arbre +
rendu des murs



Ce que l'on a prévu de faire ? - BSP

- **Ajout de textures** : (devoir lier les murs séparés, pour éviter d'avoir des textures coupées)
- **Ajout de Sprites.**
- **Ajout d'une méthode** : permettant de récupérer les murs/sprites visible pour optimiser le calcul des collisions/mouvement monstres

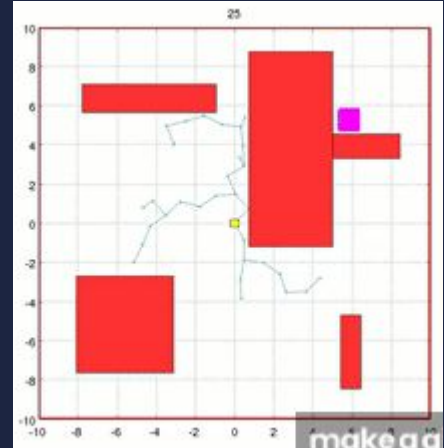


Mur coupé

Etat à la fin du premier semestre : Pathfinding

Algorithme utilisé (pour trouver le chemin) :

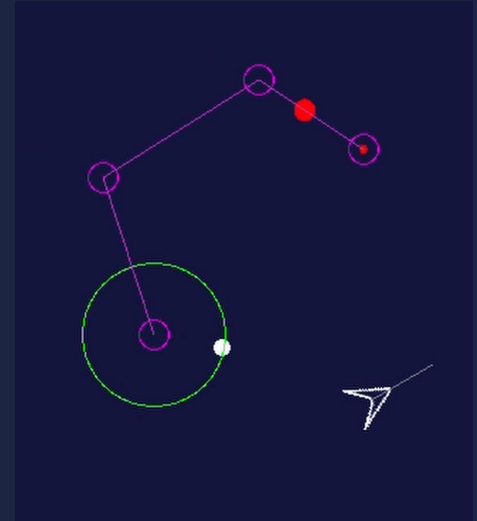
- **RRT* (Rapidly-exploring Random Tree) :**
Exploration pour des espaces plus ouverts ou complexes. Différence : A* est déterministe sur grille, RRT* est probabiliste et s'améliore avec le temps.



Source : RRT* algorithm illustrative example
<https://www.youtube.com/watch?v=YKiQTJpPFkA>

Etat à la fin du premier semestre : Steering Behaviors

- **Problème** : Les déplacements "robotiques" (virages à 90°, arrêts brusques).
- **Solution** : Implémentation de forces de pilotage (Steering Behaviors).
- **Algorithme** : $\text{Force} = \text{Vitesse Désirée} - \text{Vitesse Actuelle}$.
- **Comportements implémentés** : Seek (poursuivre), Flee (fuir), Arrival (s'arrêter doucement). Cela donne une "masse virtuelle" aux monstres.



Steering Behaviours Demonstration | Giorgio Michele de Giorgio
<https://www.giorgiomicheledegiorgio.com/projects/steering-behaviours/>

Optimisation de Steering Behavior

Optimisation RRT (Le "Buffer")

- *Problème* : L'algorithme RRT* demande énormément de calculs (ex: 10 000 itérations pour trouver un chemin optimal). Le refaire à chaque image (60 FPS) ferait planter le jeu.
- *Solution* : **Mise en cache (Buffer)**. L'arbre et le chemin sont calculés une seule fois (ou sur demande), puis le résultat (l'image du graphe) est stocké pour être réutilisé instantanément par le monstre.

Gestion des Collisions (Le "Wall Sliding")

- *Problème* : Avec le *Steering Behavior*, si le vecteur vitesse pointe vers un mur, le monstre se "colle" et s'arrête net.
- *Solution* : Algorithme de **Glissade**. Si un axe est bloqué, on conserve le mouvement sur l'autre axe (longe le mur).

Ce que l'on a prévu de faire ? - Automate à Etats Finis (FSM)

- Découpage du comportement en états distincts
- Transition basée sur des Perceptions (Vue, Dégâts)
- Les États Prévus :
 - **Patrouille** : Navigation aléatoire ou chemin prédéfini (RRT*).
 - **Poursuite** : Détection du joueur → Activation du *Seek* (Steering).
 - **Fuite** : Points de vie bas → Activation du *Flee* (Steering).

D'après Wikipédia : Un automate fini est une **construction mathématique abstraite, susceptible d'être dans un nombre fini d'états, mais étant un moment donné dans un seul état à la fois**

Architecture Réseau : Multijoueur pair-à-pair (P2P)

Rôles hybrides :

Chaque client agit simultanément comme client et serveur.

Communication :

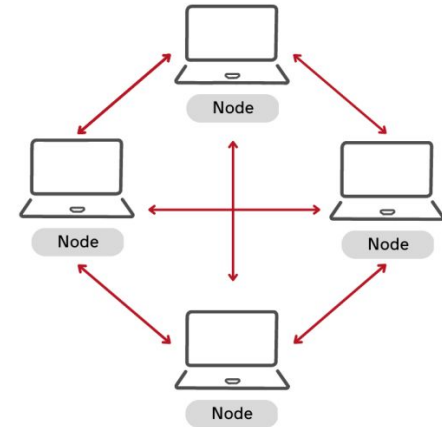
- Utilisation des classes P2PClient et P2PServer pour l'écoute et l'envoi
- Echange de messages simples contenant les coordonnées (X, Y)

Gestion de connexion :

- Système de Peer Info pour identifier les participants

CFTE

P2P Networks



Synchronisation directe entre pairs

Imprévu - réseau

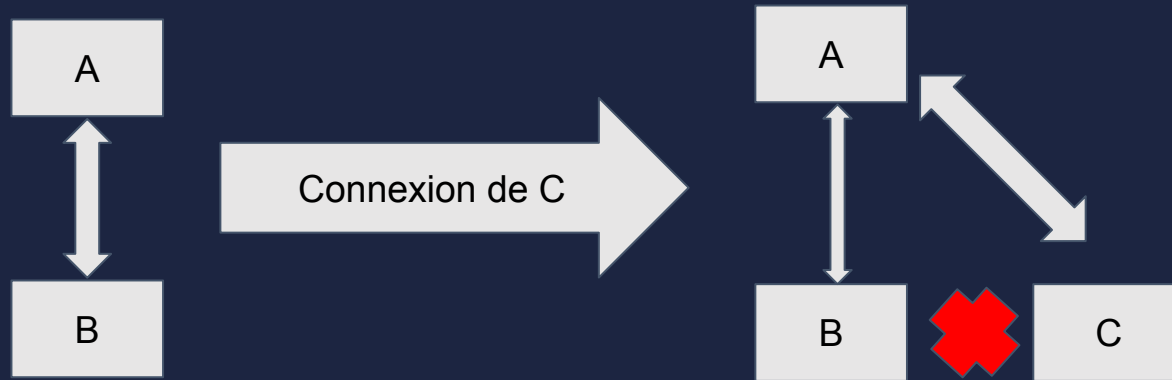
Problème en jeu :

- Tous les pairs ne se voyaient pas
- Lors de la déconnexion d'un joueur le réseau ne fonctionnait plus

Problème interne dans le serveur :

- Impossibilité d'envoyer la liste des pairs du réseau
- L'adresse IP du nouveau pair envoyé aux autres n'étaient pas la bonne.

Processus de correction de bugs réseau



Etat courant du projet VS plan initial

Réalisation :

- Raycasting et BSP implémentés avec une interface commune
- Mise en place du pathfinding avec un monstre
- Réseau P2P complètement opérationnel
- Steering behavior corrigé et réaliste.

Abandon :

- A* inadpaté avec BSP
- Behavior Tree remplacé par un automate

À quoi ressemblera le projet fini



Raycasting

Itération 4/5/6

BSP + raytracing + environnement 3D + open world + automate de comportement de groupes



Questions ?



Merci pour votre attention